

# AI\_PROJECT\_CONTEXT.md

## VSSC ISRO Infrastructure Management Portal

### Master AI Context Document

Version: 1.0

Last Updated: June 2026

---

## PURPOSE OF THIS DOCUMENT

This document serves as the primary knowledge source for any AI assistant, developer, architect, or future contributor working on the VSSC Infrastructure Management Portal frontend redesign project.

Before making any modification to the codebase, this document should be read completely.

This document defines:

- Project goals
- Existing system architecture
- Design philosophy
- Constraints
- Role model
- Current implementation status
- Development roadmap
- Rules for future development

The goal is to ensure that future development remains consistent even when performed by different developers or AI systems.

---

## PROJECT OVERVIEW

### Project Name

VSSC Infrastructure Management Portal

(Internally derived from the existing VMDash platform)

---

## Organization

Vikram Sarabhai Space Centre (VSSC)

Indian Space Research Organisation (ISRO)

---

## Project Type

Internal Infrastructure Operations Platform

---

## Primary Objective

The objective of this project is to redesign and modernize the frontend of the existing VM Infrastructure Management System without changing:

- Database schema
- Backend APIs
- Authentication architecture
- Existing business workflows

The project is a frontend modernization effort, not a backend redevelopment effort.

---

## EXISTING SYSTEM SUMMARY

The existing application acts as an operational portal for infrastructure administrators managing Proxmox virtualization resources.

The system provides visibility into:

- Virtual Machines
- Physical Nodes
- Hypervisor Clusters
- Storage Resources
- VM Ownership
- Staff Allocation
- Capacity Monitoring
- Infrastructure Synchronization

The application consumes data from:

- proxmox\_db
- ccds\_db

and two Flask-based backend services.

---

## CURRENT PROBLEM STATEMENT

The original system suffers from several issues:

### UI Problems

- Outdated visual design
  - Generic administrative appearance
  - Poor information hierarchy
  - Limited visualizations
  - Text-heavy screens
  - Weak dashboard experience
- 

### UX Problems

- Navigation fragmentation
  - Inconsistent workflows
  - Poor role differentiation
  - Limited feedback during operations
  - Difficult infrastructure monitoring
- 

### Technical Problems

- Limited frontend structure
  - Weak component reusability
  - Inconsistent styling
  - Minimal animation support
  - Difficult future extensibility
- 

## PROJECT VISION

Transform the platform from:

Basic Infrastructure CRUD Dashboard

into

## Infrastructure Operations Center

The final application should help users answer:

- What infrastructure exists?
- Who owns it?
- How much capacity is allocated?
- What resources are growing?
- What needs attention?
- What trends are emerging?

The system should feel closer to an operational command center than a traditional admin panel.

---

# ABSOLUTE CONSTRAINTS

These rules must never be violated.

## Database Constraints

Do not:

- Add tables
- Modify tables
- Remove tables
- Change relationships

All functionality must work with the existing schema.

---

## Backend Constraints

Do not:

- Add new API endpoints
- Change endpoint contracts
- Modify authentication mechanisms

Frontend development must consume existing APIs only.

---

## Workflow Constraints

Do not invent business processes.

Every workflow must be traceable to:

- Existing APIs
  - Existing database entities
  - Existing backend functionality
- 

## **Analytics Constraints**

Analytics must be derived from existing data.

Allowed:

- Aggregations
- Trends
- Historical comparisons
- Capacity estimates

Not allowed:

- AI forecasting claims
  - Machine learning predictions without backend support
- 

## **TECHNOLOGY STACK**

### **Frontend**

React

React Router

Axios

Tailwind CSS

Framer Motion

Lucide React

Plotly

---

## Backend

Flask

Python

REST APIs

---

## Databases

MySQL

proxmox\_db

ccds\_db

---

## Infrastructure

Proxmox

Nginx

RHEL 9.x

---

# DESIGN SYSTEM

## Design Philosophy

The application should feel:

Professional

Institutional

Technical

Reliable

Modern

Information-Dense

Not flashy.

Not consumer-oriented.

Not startup-themed.

---

## COLOR SYSTEM

### Light Theme

Background:

**F8FAFC**

Surface:

**FFFFFF**

Primary:

**1D4ED8**

Accent:

**2563EB**

Text:

**0F172A**

Border:

**E2E8F0**

---

## Dark Theme

Background:

**0B1220**

Surface:

**111827**

Primary:

**2563EB**

Accent:

**3B82F6**

Text:

**F8FAFC**

Border:

**1F2937**

---

## TYPOGRAPHY

Primary Font:

Noto Serif

Fallback:

Liberation Serif

Final Fallback:



Times New Roman

Reason:

- Enterprise Linux compatibility
  - RHEL compatibility
  - Professional appearance
  - Readability in dense dashboards
- 

## **COMPONENT STANDARDS**

### **Cards**

Rounded corners

16px–18px radius

Soft shadows

Theme aware

---

### **Tables**

Sticky headers

Hover states

Dense but readable

Responsive

Theme aware

---

### **Forms**

Consistent spacing

Validation feedback

Accessible controls

---

## Icons

Lucide React only

Avoid mixing icon libraries.

---

## ANIMATION SYSTEM

Use Framer Motion.

Allowed:

- Fade
- Slide
- Scale
- Stagger

Required:

- Page transitions
- Card entrances
- Drawer animations
- Modal animations
- Sidebar transitions

Avoid excessive effects.

Animations should feel deliberate and professional.

---

## ROLE MODEL

Three roles exist.

---

## ADMIN

Full access.

Can:

- View all infrastructure
  - Sync resources
  - Manage users
  - Manage VM assignments
  - Create VM requests
  - Generate reports
  - Access analytics
  - Access monitoring
- 

## MANAGER

Operational observer.

Can:

- View infrastructure
- View VMs
- View dashboards
- View monitoring
- View analytics
- Generate reports

Cannot:

- Sync
  - Create users
  - Create VM requests
  - Modify assignments
- 

## USER

Restricted visibility.

Can only access resources assigned through:

vm\_user\_relation

Users may access:

- My Dashboard
- My Virtual Machines

- My Analytics
- My Reports
- Settings

Users may not access:

- Global infrastructure
  - Nodes
  - Clusters
  - Storage inventories
  - Administrative functionality
- 

## APPLICATION STRUCTURE

### Dashboard

Primary landing page.

Provides infrastructure overview.

---

### Infrastructure

Clusters

Nodes

Storage

---

### Virtual Machines

VM Inventory

VM Details

Ownership

Storage Mapping

---

## Monitoring

Pressure Analytics

Resource Trends

---

## Analytics

Entity Usage

Division Usage

Capacity Projection

---

## Reports

Report Builder

Report History

Exports

---

## Administration

User Management

Sync Center

VM Request Management

---

## Settings

Profile

Theme

Password

---

# CURRENT IMPLEMENTATION STATUS

## Completed

### Stage 0

System Reverse Engineering

Completed

---

### Stage 1

Foundation Refactor

Completed

Includes:

- Application shell
  - Sidebar
  - Topbar
  - Theme system
  - Routing structure
  - Motion foundation
  - Design tokens
- 

### Stage 1.1

Backend Integration Pass

Completed

Includes:

- Nodes integration
  - Cluster integration
  - Storage integration
  - Metrics integration
- 

### Stage 1.2

Analytics Foundation

Completed

Includes:

- Entity usage analytics
  - Division usage analytics
  - Trend visualizations
  - Capacity projections
- 

### **Stage 1.3**

RBAC Foundation

Completed

Includes:

- Role-aware navigation
  - User filtering
  - Protected routes
- 

## **CURRENT STAGE**

Stage 1.4

Role Validation & Sync Verification

Objectives:

- Verify sync workflows
  - Verify access boundaries
  - Verify user visibility rules
  - Verify route protection
- 

## **FUTURE DEVELOPMENT ROADMAP**

### **Stage 2**

Dashboard Refinement

Infrastructure Dashboards

Clusters

Nodes

Storage

---

## Stage 3

VM Management Redesign

Advanced table

Drawer-based details

Ownership tab

Storage tab

Technical details tab

---

## Stage 4

Monitoring Center

Pressure Analytics

Resource Trends

Operational monitoring

---

## Stage 5

Advanced Analytics

Entity analytics

Division analytics

Capacity projections

Comparative views



---

## Stage 6

Reports & Administration

Report Builder

Sync Center

User Management

Activity Center

---

## Stage 7

User Experience Refinement

My Dashboard

My VMs

My Analytics

My Reports

---

## Stage 8

Final Polish

Accessibility

Performance

Cross-browser testing

RHEL validation

Animation refinement

Loading states

Error states

## DEVELOPMENT RULES

Every future implementation must:

1. Read this document.
  2. Respect database constraints.
  3. Respect API constraints.
  4. Preserve existing workflows.
  5. Follow design system guidelines.
  6. Maintain role boundaries.
  7. Use reusable components.
  8. Preserve theme compatibility.
  9. Preserve accessibility.
  10. Maintain consistency with roadmap.
- 

## SUCCESS CRITERIA

The redesign is considered successful when:

- Infrastructure visibility is significantly improved.
- Navigation is intuitive.
- Analytics are meaningful.
- User roles are clearly separated.
- Visual design is modern and professional.
- Existing backend functionality remains fully operational.
- The system feels like an Infrastructure Operations Center rather than a basic CRUD dashboard.